Cicli: esercizi

Dario Pescini

Università degli Studi di Milano-Bicocca

Dipartimento di Statistica e Metodi Quantitativi

dario.pescini@unimib.it

variabili di accumulo: media

Spesso si utilizzano i cicli per “accumulare” quantità variabili in una variabile detta d’accumulo o accumulatore.

```
numIterazioni = 5
media = 0
contatore = 0
while contatore < numIterazioni:
    valore = float(input('inserisci un numero '))
    media += valore
    contatore += 1

media /= numIterazioni
print('il valor medio è ', media)
```

Esercizio - varianza

Scrivere un algoritmo per calcolare media e varianza di 10 valori inseriti da tastiera dall'utente.

Esercizio - varianza

Scrivere un algoritmo per calcolare media e varianza di 10 valori inseriti da tastiera dall'utente.

- media: $\mu = \frac{\sum_{i=1}^{N} x_i}{N}$
- varianza della popolazione: $\sigma^2 = \frac{\sum_{i=1}^{N} (x_i - \mu)^2}{N}$

Esercizio - varianza

Scrivere un algoritmo per calcolare media e varianza di 10 valori inseriti da tastiera dall'utente.

- media: $\mu = \frac{\sum_{i=1}^{N} x_i}{N}$
- varianza della popolazione: $\sigma^2 = \frac{\sum_{i=1}^{N} (x_i - \mu)^2}{N}$
- varianza campionaria, nella forma di calcolo: $s^2 = \frac{\sum_{i=1}^{N} x_i^2 - \frac{(\sum_{i=1}^{N} x_i)^2}{N}}{N-1}$

Esercizio - varianza

Scrivere un algoritmo per calcolare media e varianza di 10 valori inseriti da tastiera dall'utente.

- media: $\mu = \frac{\sum_{i=1}^{N} x_i}{N}$
- varianza della popolazione: $\sigma^2 = \frac{\sum_{i=1}^{N} (x_i - \mu)^2}{N}$
- varianza campionaria, nella forma di calcolo: $s^2 = \frac{\sum_{i=1}^{N} x_i^2 - \frac{(\sum_{i=1}^{N} x_i)^2}{N}}{N-1}$
- **non sono la stessa cosa**: su 1, 2, ..., 10 danno `8.25` e `9.17`. Il programma che segue calcola s^2
- la forma di calcolo ha un vantaggio per un ciclo: basta **accumulare** $\sum x_i$ e $\sum x_i^2$, senza ricordare i valori inseriti

Esercizio

```
N = 10



somma = 0.0
somma_quadrati = 0.0

contatore = 0
while contatore < N:
    numero = input('inserisci un numero ')
    print("iterazione ", contatore)
    somma += float(numero)
    somma_quadrati += float(numero)**2
    contatore += 1

media = somma / N
varianza = (somma_quadrati - (somma**2 / N)) / (N - 1.0)

print("media = ", media)
print("varianza campionaria = ", varianza)
```

Esercizio

Si costruisca un programma che ricevuto un numero intero riconosca se è primo.

Esercizio

Si costruisca un programma che ricevuto un numero intero riconosca se è primo.

Un numero naturale, maggiore di 1, si dice primo se è divisibile solo per se stesso e per l'unità.

Esercizio

Si costruisca un programma che ricevuto un numero intero riconosca se è primo.

Un numero naturale, maggiore di 1, si dice primo se è divisibile solo per se stesso e per l'unità.

Come posso verificarlo?

Esercizio

Si costruisca un programma che ricevuto un numero intero riconosca se è primo.

Un numero naturale, maggiore di 1, si dice primo se è divisibile solo per se stesso e per l'unità.

Come posso verificarlo?

- un numero *n* è divisibile per un divisore *d* se
  $n \mod d = 0$ o equivalentemente *resto* $= 0$

Esercizio

Si costruisca un programma che ricevuto un numero intero riconosca se è primo.

Un numero naturale, maggiore di 1, si dice primo se è divisibile solo per se stesso e per l'unità.

Come posso verificarlo?

- un numero *n* è divisibile per un divisore *d* se $n \mod d = 0$ o equivalentemente *resto* $= 0$
- ha come unici divisori 1 e se stesso o equivalentemente l'insieme dei divisori $\mathcal{D} \subseteq \mathbb{N}$ deve essere $\mathcal{D} = \{1, n\}$

Esercizio

Si costruisca un programma che ricevuto un numero intero riconosca se è primo.

Un numero naturale, maggiore di 1, si dice primo se è divisibile solo per se stesso e per l'unità.

Come posso verificarlo?

- un numero *n* è divisibile per un divisore *d* se $n \mod d = 0$ o equivalentemente *resto* $= 0$
- ha come unici divisori 1 e se stesso o equivalentemente l'insieme dei divisori $\mathcal{D} \subseteq \mathbb{N}$ deve essere $\mathcal{D} = \{1, n\}$
- posso generare tutti i numeri $d \in \mathbb{N} \mid d \leq n$ e verificare per ciascuno se *d* è divisore

Esercizio

Si costruisca un programma che ricevuto un numero intero riconosca se è primo.

Un numero naturale, maggiore di 1, si dice primo se è divisibile solo per se stesso e per l'unità.

Come posso verificarlo?

- un numero *n* è divisibile per un divisore *d* se $n \mod d = 0$ o equivalentemente *resto* $= 0$
- ha come unici divisori 1 e se stesso o equivalentemente l'insieme dei divisori $\mathcal{D} \subseteq \mathbb{N}$ deve essere $\mathcal{D} = \{1, n\}$
- posso generare tutti i numeri $d \in \mathbb{N} \mid d \leq n$ e verificare per ciascuno se *d* è divisore
- basta provare i *d* da 2 a $n - 1$: al primo che divide *n*, *n* non è primo, e si può smettere di cercare

Esercizio: soluzione

```
n = int(input("un numero intero: "))

primo = n > 1
d = 2
while d < n and primo:
    if n % d == 0:
        primo = False
    d = d + 1

if primo:
    print(n, "è primo")
else:
    print(n, "non è primo")
```

- `primo` è un **flag**: parte vero, diventa falso al primo divisore trovato
- `and primo` nella condizione **ferma il ciclo** appena il flag cade: inutile cercare altri divisori

Esercizio: soluzione

```
n = int(input("un numero intero: "))

primo = n > 1
d = 2
while d < n and primo:
    if n % d == 0:
        primo = False
    d = d + 1

if primo:
    print(n, "è primo")
else:
    print(n, "non è primo")
```

- `primo` è un **flag**: parte vero, diventa falso al primo divisore trovato
- `and primo` nella condizione **ferma il ciclo** appena il flag cade: inutile cercare altri divisori
- `primo = n > 1`: 0 e 1 non sono primi, e per 2 il ciclo non gira nemmeno

Si può fare più veloce?

```
n = int(input("un numero intero: "))

if n < 2:
    primo = False
elif n % 2 == 0:
    primo = n == 2
else:
    primo = True
    d = 3
    while d * d <= n and primo:
        if n % d == 0:
            primo = False
        d = d + 2
```

Due idee, e nessuna riga di Python nuova:

- **i pari si decidono subito**: l'unico primo pari è 2. Poi si provano **solo i divisori dispari**, `d = d + 2`

Si può fare più veloce?

```
n = int(input("un numero intero: "))

if n < 2:
    primo = False
elif n % 2 == 0:
    primo = n == 2
else:
    primo = True
    d = 3
    while d * d <= n and primo:
        if n % d == 0:
            primo = False
        d = d + 2
```

Due idee, e nessuna riga di Python nuova:

- **i pari si decidono subito**: l'unico primo pari è 2. Poi si provano **solo i divisori dispari**, `d = d + 2`
- **basta arrivare a $\sqrt{n}$**: se $n = a \cdot b$, uno dei due non supera $\sqrt{n}$. Se fin lì non c'è un divisore, non ce n'è nessuno
- `d * d <= n` dice la stessa cosa di $d \leq \sqrt{n}$, senza calcolare radici

Un problema, più algoritmi

Quanti giri fa il ciclo per riconoscere un numero primo:

algoritmo	97	9 973	999 983
tutti i divisori fino a n	95	9 971	999 981
solo i dispari fino a n	48	4 986	499 991
tutti i divisori fino a $\sqrt{n}$	8	98	998
solo i dispari fino a $\sqrt{n}$	4	49	499

- **quattro algoritmi, la stessa risposta**: sono tutti corretti

Un problema, più algoritmi

Quanti giri fa il ciclo per riconoscere un numero primo:

algoritmo	97	9 973	999 983
tutti i divisori fino a n	95	9 971	999 981
solo i dispari fino a n	48	4 986	499 991
tutti i divisori fino a $\sqrt{n}$	8	98	998
solo i dispari fino a $\sqrt{n}$	4	49	499

- **quattro algoritmi, la stessa risposta**: sono tutti corretti
- saltare i pari **dimezza** i giri; fermarsi a $\sqrt{n}$ li divide per mille. Conta più **pensare al problema** che scrivere codice veloce
- prima si scrive un programma **corretto e leggibile**, poi, se serve, uno più veloce: è la distinzione fra **algoritmo** e **programma** di `L03`

Tutti i primi fino a N

```
N = int(input("fino a quale numero? "))

num = 2
while num <= N:
    primo = True
    d = 2
    while d < num and primo:
        if num % d == 0:
            primo = False
        d = d + 1
    if primo:
        print(num)
    num = num + 1
```

- il ciclo **interno** è il programma di prima, **intero**
- quello **esterno** lo ripete per ogni `num` da 2 a `N`

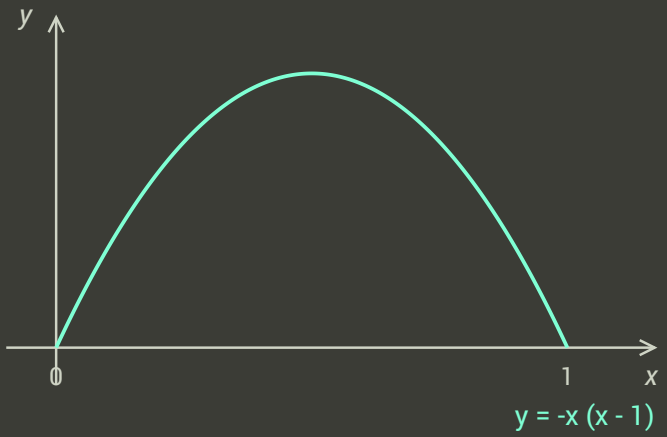
Tutti i primi fino a N

```
N = int(input("fino a quale numero? "))

num = 2
while num <= N:
    primo = True
    d = 2
    while d < num and primo:
        if num % d == 0:
            primo = False
        d = d + 1
    if primo:
        print(num)
    num = num + 1
```

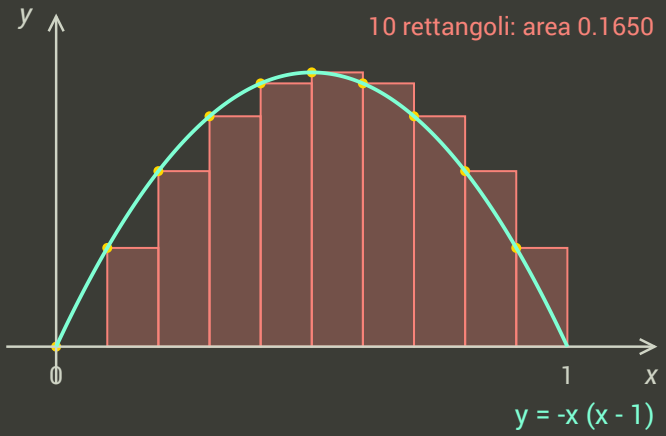
- il ciclo **interno** è il programma di prima, **intero**
- quello **esterno** lo ripete per ogni `num` da 2 a `N`
- `primo = True` e `d = 2` stanno dentro il ciclo esterno: ogni numero riparte da capo, come `j = 1` nelle tabelline

Esercizio



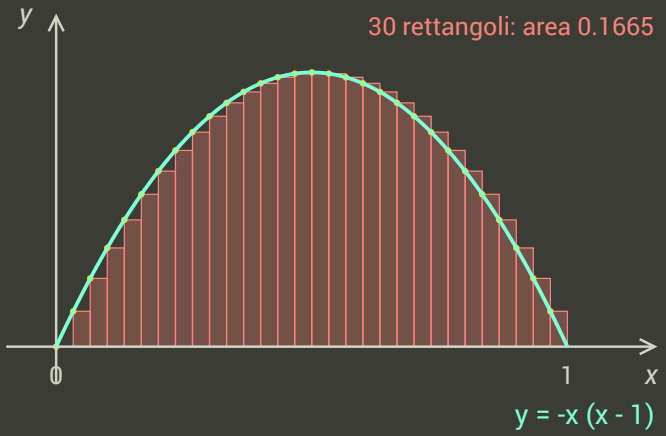
Si calcoli l'area sottesa dalla parabola in figura.

Esercizio



Si calcoli l'area sottesa dalla parabola in figura.

Esercizio



Si calcoli l'area sottesa dalla parabola in figura.

Esercizio: decomporre il problema in problemi più semplici

- L'area sottesa può essere approssimata come la somma delle aree di rettangoli contigui aventi per base l'asse delle ascisse e per altezza il punto di contatto con la parabola

Esercizio: decomporre il problema in problemi più semplici

- L'area sottesa può essere approssimata come la somma delle aree di rettangoli contigui aventi per base l'asse delle ascisse e per altezza il punto di contatto con la parabola

Esercizio: decomporre il problema in problemi più semplici

- L'area sottesa può essere approssimata come la somma delle aree di rettangoli contigui aventi per base l'asse delle ascisse e per altezza il punto di contatto con la parabola
- costruire i rettangoli

Esercizio: decomporre il problema in problemi più semplici

- L'area sottesa può essere approssimata come la somma delle aree di rettangoli contigui aventi per base l'asse delle ascisse e per altezza il punto di contatto con la parabola
- costruire i rettangoli
- determinare base e altezza di un rettangolo

Esercizio: decomporre il problema in problemi più semplici

- L'area sottesa può essere approssimata come la somma delle aree di rettangoli contigui aventi per base l'asse delle ascisse e per altezza il punto di contatto con la parabola
- costruire i rettangoli
- determinare base e altezza di un rettangolo
- calcolare area di un rettangolo

Esercizio: decomporre il problema in problemi più semplici

- L'area sottesa può essere approssimata come la somma delle aree di rettangoli contigui aventi per base l'asse delle ascisse e per altezza il punto di contatto con la parabola
- costruire i rettangoli
- determinare base e altezza di un rettangolo
- calcolare area di un rettangolo
- sommare i contributi

Esercizio: decomporre il problema in problemi più semplici

- per costruire la base dei rettangoli si può suddividere l'intervallo $I = [0, 1]$ in un numero finito, M, di intervalli contigui I_i.
- in questo caso ciascun intervallo I_i avrà lunghezza pari a $\frac{1}{M}$ (base del rettangolo)
- ciascun rettangolo (i) avrà per base il segmento $[\frac{i}{M}, \frac{i+1}{M}]$
- l'altezza potrebbe essere $y_i = -x_i(x_i - 1) = -\frac{i}{M}(\frac{i}{M} - 1)$

Esercizio: soluzione

```
numIntervalli = int(input('inserire il numero di intervalli in [0.0, 1.0] '))

deltaIntervallo = 1.0 / float(numIntervalli)
print("larghezza intervallo", deltaIntervallo)

areaSottesa = 0.0
i = 0
while i < numIntervalli:
    xIntervallo = i * deltaIntervallo
    yIntervallo = - xIntervallo * (xIntervallo - 1.0)
    areaSottesa += yIntervallo * deltaIntervallo
    i += 1

print("l'area sottesa dalla curva vale ", areaSottesa)
```

Quanto è precisa?

intervalli	area calcolata
10	`0.165`
100	`0.16665`
1000	`0.1666665`
esatta	$\frac{1}{6} = 0.1\overline{6}$

- più rettangoli, più stretti: l'approssimazione **migliora**, ma non arriva mai al valore esatto
- i rettangoli prendono l'altezza dal **lato sinistro**: è una somma di Riemann, e il calcolo dell'integrale la porta al limite